



**COMSATS University Islamabad,
Abbottabad Campus**

SOFTWARE REQUIREMENTS SPECIFICATION

(SRS DOCUMENT)

for

**SmartCrop – AI & IoT-Based Crop Suitability and
Monitoring System**

Version 1.0

By

Aleeza Javed

CIIT/SP23-BCS-141/ATD

Zulkifl Ahmad

CIIT/SP23-BCS-047/ATD

Muneeba Kalsoom

CIIT/SP23-BCS-147/ATD

Supervisor

Sir Yasar Khan

Bachelor of Science in Computer Science (2023-2027)

Table of Contents

Introduction	1
Purpose.....	1
Scope.....	1
Overall description	1
Product perspective.....	1
Operating environment.....	2
Design and implementation constraints	2
Requirement identifying technique.....	2
Use case diagram	3
Use case description.....	3
Functional Requirements.....	7
Non Functional Requirements.....	14
References	16

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by
<Supervisor's Name>

Signature_____

Introduction

SmartCrop is an AI and IoT-based intelligent agricultural system designed to help farmers monitor soil conditions and make data-driven decisions. The system collects real-time environmental data using sensors such as soil moisture, temperature, humidity, pH, and nutrient levels. This data is transmitted to a cloud database (Firebase), where it is stored and processed. A machine learning model analyzes the collected data and recommends the most suitable crop for cultivation. The system also provides smart alerts, automated irrigation control, and a mobile application interface for remote monitoring. The main purpose of SmartCrop is to improve crop productivity, reduce water wastage, and modernize traditional farming practices using advanced technology.

Purpose

This document specifies the software requirements for SmartCrop, an AI and IoT-based intelligent farming system designed to help farmers monitor soil conditions and get crop recommendations.

Scope

SmartCrop is an AI and IoT-based intelligent system designed to assist farmers in making data-driven decisions regarding crop selection and field monitoring. The system collects real-time soil (pH, NPK, moisture, temperature) and environmental (humidity, rainfall, light intensity) data via an ESP32-based sensor array. This data is transmitted to a Firebase cloud backend. A machine learning model (Scikit-learn/TensorFlow) analyzes the data to recommend the most suitable crop among five major crops (Wheat, Rice, Maize, Cotton, Tomato). The system also features automated irrigation control and a Flutter-based Android mobile application that provides a real-time dashboard, smart alerts, and data visualization. This release focuses on a controlled environment prototype and does not include disease detection, pest identification, or a web-based dashboard.

Overall description

Product perspective

SmartCrop is a new, standalone, self-contained system designed to modernize traditional farming. It is not a replacement for an existing system but rather an innovative solution that integrates IoT hardware, cloud computing, and AI/ML. It acts as a decision-support tool that sits between the farmer and the field, providing actionable insights. The system is composed of four main subsystems:

- IoT Sensor & Actuator Subsystem (ESP32, sensors, water pump relay)
- Cloud Backend Subsystem (Firebase Realtime Database, Authentication, FCM)
- AI/ML Prediction Subsystem (Python-based model)

- Mobile Application Subsystem (Flutter/Dart Android App)

Operating environment

The system will operate in the following environment:

- **IoT Device:** ESP32 microcontroller placed in an agricultural field (initially a controlled environment testbed) within WiFi range.
- **Sensors:** Soil pH, NPK, moisture, temperature, and humidity sensors connected to the ESP32.
- **Backend:** Firebase cloud platform (Google servers).
- **Mobile App:** Android OS (version 10 or higher) on smartphones used by farmers.
- **Connectivity:** Stable 4G/WiFi internet connection required for the ESP32 and the mobile app to communicate with Firebase. No offline mode for AI predictions.
- **Power:** Continuous AC power supply for the prototype; a 5V/2A USB power adapter for the ESP32.

Design and implementation constraints

- **CO-1:** The system shall be developed using only open-source or freely available tools: Arduino IDE, VS Code, Flutter SDK, Python, and Firebase (Spark free tier).
- **CO-2:** The AI model shall initially be trained on the public "Crop Recommendation Dataset" from Kaggle, as local Pakistani datasets are not readily available.
- **CO-3:** The system shall support only five (5) pre-defined crops: Wheat, Rice, Maize, Cotton, and Tomato.
- **CO-4:** The system shall not include any image processing for disease or pest detection.
- **CO-5:** The automatic irrigation control shall be limited to a single water pump relay for a small-scale prototype (e.g., a pot or small plot).
- **CO-6:** The system shall require a continuous internet connection for the ESP32 to send data and for the mobile app to receive real-time updates and AI predictions. Offline AI prediction is not supported.

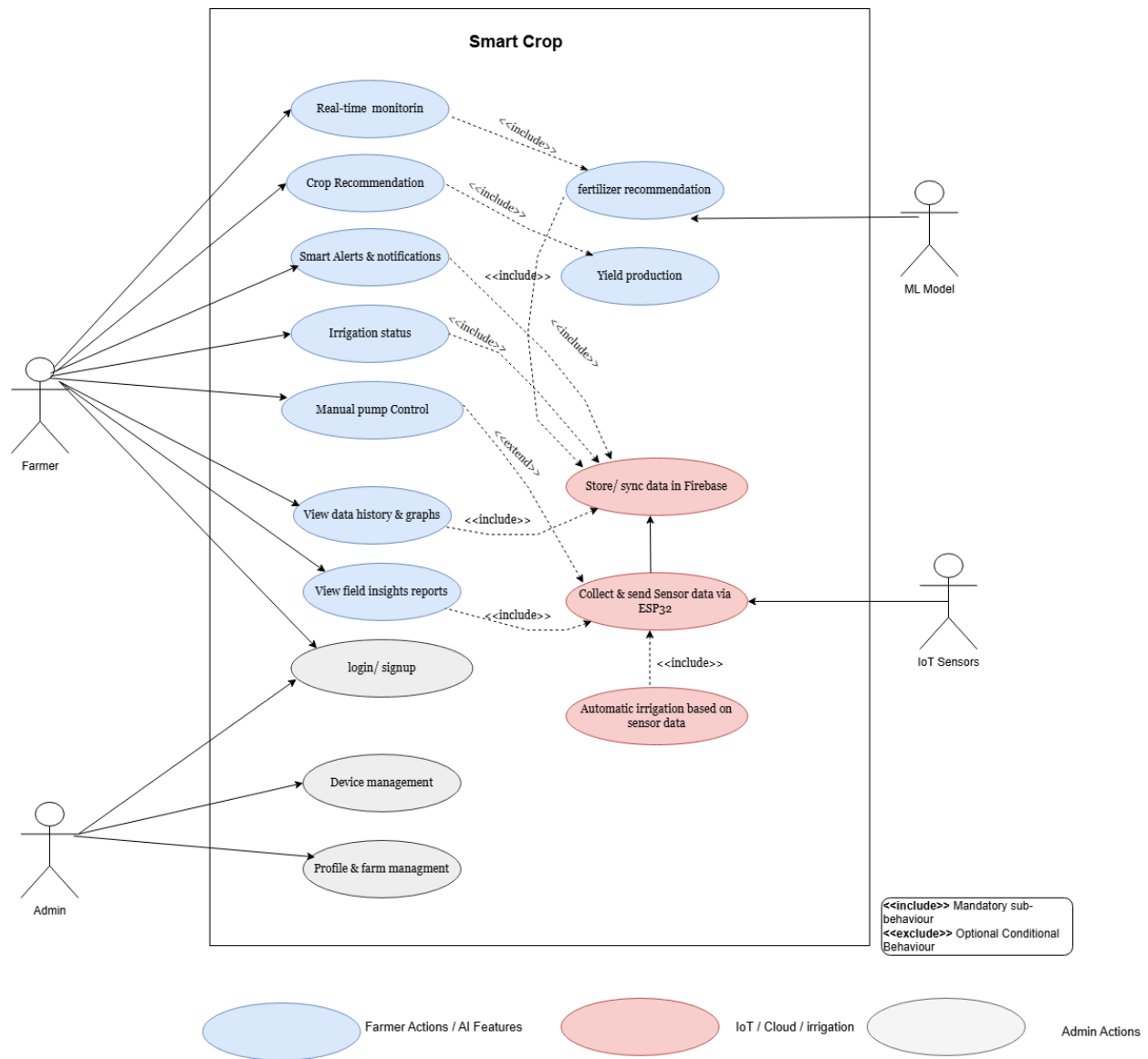
Requirement identifying technique

This section describes the requirements identifying technique(s) which further help to derive functional requirements specification. The selection of the technique(s) will depend on the type of project. For instance,

- **Use case** is an effective technique for interactive end-user applications
- **Event- response tables** is for real time system and
- **Story boarding** for graphically intensive applications.

In addition to above, the projects involving data warehouses, batch processes, hardware devices with embedded control software, and computationally intensive applications required to follow other suitable techniques. Such techniques are described further in Chapter 12, "A picture is worth 1024 words." For documenting this section let consider identifying requirements through use case as an example.

Use case diagram



Use case description

Table 1 Show the detail use case template

Use case 1

Use Case ID:	UC-01
Use Case Name:	Login / Signup
Actors:	Primary Actor: Farmer Secondary Actors: FirebaseAuthentication System,

Description:	A user (farmer or admin) creates an account or logs into the SmartCrop system to access dashboard features like monitoring, recommendations, and reports.
Trigger:	User selects “Login” or “Signup” option.
Preconditions:	PRE-1. User must have a valid internet connection PRE-2. User must be registered (for login only)
Postconditions:	POST-1. User is authenticated successfully. POST-2. User is redirected to dashboard.
Normal Flow:	1. User opens app 2. System displays login/signup screen 3. User enters credentials 4. System verifies data via Firebase 5. System authenticates user 6. Dashboard is displayed
Alternative Flows: [Alternative Flow 1 – Not in Network]	A1: User selects signup → system creates new account → redirects to login/dashboard
Exceptions:	E1: Invalid credentials then system shows error message E2: No internet connection then login fails
Business Rules	BR-1 Only registered users can access system features
Assumptions:	Users have basic mobile literacy and internet access

Use case 2

Use Case ID:	UC-02
Use Case Name:	Real-Time Monitoring
Actors:	Primary Actor: Farmer Secondary Actors: IoT Sensors, Firebase
Description:	The system displays real-time soil and environmental data such as temperature, humidity, and soil moisture collected from IoT sensors.
Trigger:	Farmer opens “Real-time Monitoring” dashboard.
Preconditions:	PRE-1. Sensors must be active PRE-2. Data must be syncing to Firebase
Postconditions:	POST-1. Live sensor data is displayed on dashboard
Normal Flow:	1. Farmer opens monitoring screen 2. System requests latest sensor data 3. IoT sensors send data via ESP32 4. Firebase stores data 5. App retrieves and displays data in real-time
Alternative Flows:	A1: Data refresh manually triggered by user

[Alternative Flow 1 – Not in Network]	
Exceptions:	E1: If sensor is offline then system will shows last updated data E2: If Firebase sync failure then error message displayed
Business Rules	BR-1 Data refresh interval = real-time (or every 5–10 seconds)
Assumptions:	Sensors are installed properly in field

Use case 3

Use Case ID:	UC-03
Use Case Name:	Crop Recommendation
Actors:	Primary Actor: Farmer Secondary Actors: ML Model
Description:	System recommends suitable crops based on soil and environmental data using machine learning analysis.
Trigger:	Farmer selects “Crop Recommendation” option.
Preconditions:	PRE-1. Sensor data must be available
Postconditions:	POST-1. Recommended crops are displayed
Normal Flow:	1. Farmer requests recommendation 2. System collects soil & weather data 3. Data is sent to ML model 4. Model processes input 5. System returns best crop suggestions 6. Results are displayed to farmer
Alternative Flows: [Alternative Flow 1 – Not in Network]	A1: Data refresh manually enter soil values
Exceptions:	E1: ML model failure → system shows default recommendation
Business Rules	BR-1 Recommendation is based on trained dataset accuracy
Assumptions:	ML model is pre-trained and deployed

Use Case 4

Use Case ID:	UC-04
Use Case Name:	Irrigation Status & Control
Actors:	Primary Actor: Farmer Secondary Actors: IoT Sensors, ESP32 Controller

Description:	System shows irrigation status and allows manual/automatic pump control based on soil moisture data.
Trigger:	Farmer opens irrigation module.
Preconditions:	PRE-1. Pump system is connected PRE-2. Sensor data is available
Postconditions:	POST-1. Irrigation system state is updated
Normal Flow:	1. Farmer opens irrigation screen 2. System shows soil moisture level 3. System shows pump status 4. Farmer selects manual/auto mode 5. ESP32 executes command 6. Status updated in system
Alternative Flows: [Alternative Flow 1 – Not in Network]	A1: Automatic irrigation triggered by low moisture
Exceptions:	E1: Pump failure then system shows alert
Business Rules	BR-1 Irrigation starts when soil moisture < threshold
Assumptions:	Hardware system is functional

Use case 5

Use Case ID:	UC-05
Use Case Name:	Smart Alerts & Notifications
Actors:	Primary Actor: Farmer Secondary Actors: Firebase Cloud Messaging
Description:	System sends alerts about weather changes, irrigation needs, or crop conditions.
Trigger:	Sensor data crosses threshold values.
Preconditions:	PRE-1. Notification service enabled
Postconditions:	POST-1. User receives alert notification
Normal Flow:	1. System monitors sensor data 2. Condition threshold is met 3. Alert is generated 4. Notification sent to farmer 5. Farmer views alert
Exceptions:	E1: Notification service down → alert stored in system
Business Rules	BR-1 Alerts must be sent within 1 minute of event

Use case 6

Use Case ID:	UC-06
Use Case Name:	Store / Sync Data in Firebase
Actors:	Primary Actor: IoT Sensors Secondary Actors: Firebase Database
Description:	Sensor data is collected and stored in Firebase for real-time access and analysis.
Trigger:	Sensor sends new data packet.
Preconditions:	PRE-1. Internet connection available
Postconditions:	POST-1. Data stored successfully in database
Normal Flow:	1. Sensors collect field data 2. ESP32 sends data 3. Firebase receives data 4. Data is stored and updated 5. App retrieves updated values
Exceptions:	E1: Network failure then data buffered locally
Business Rules	BR-1 Data must be updated continuously in real-time

Functional Requirements

Module 1: IoT Sensor Data Collection

Table 2 Show the functional requirement template

Identifier	FR-01
Title	Real-time Soil Parameter Measurement
Requirement	The system shall use calibrated sensors connected to an ESP32 microcontroller to measure soil pH, Nitrogen (N), Phosphorus (P), Potassium (K), and soil moisture every 30 minutes.
Source	Aleeza Javed (IoT Module)
Rationale	To provide the fundamental input data for all prediction models and irrigation control.
Dependencies	None
Priority	High

Identifier	FR-02
Title	Real-time Environmental Parameter Measurement
Requirement	The system shall use environmental sensors (DHT22 for temperature/humidity, rain sensor, and light sensor) connected to the ESP32 to measure ambient temperature (°C), relative humidity (%), rainfall status (Yes/No), and light intensity (Lux) every 30 minutes simultaneously with soil parameters.
Source	Aleeza Javed (IoT Module)
Rationale	Environmental factors significantly influence crop growth and yield; integrating these parameters improves the accuracy of AI predictions.
Dependencies	FR-01
Priority	High
Identifier	FR-03
Title	Sensor Calibration and Accuracy Validation
Requirement	The system shall include a calibration mechanism for each analog sensor (pH, NPK, moisture) using standard reference solutions. The ESP32 firmware shall compute and apply calibration offsets before transmitting data to ensure accuracy within $\pm 5\%$ of reference values.
Source	Aleeza Javed (IoT Module)
Rationale	Uncalibrated sensors produce unreliable data, leading to incorrect crop recommendations and irrigation decisions.
Dependencies	FR-01
Priority	Medium

Module 2: Firebase Cloud Data Logging Module

Identifier	FR-04
Title	Cloud Data Storage and Synchronization
Requirement	The ESP32 shall send all sensor readings to the Firebase Realtime Database immediately after each reading. Each entry shall be timestamped and structured under a unique user/device ID. The mobile app shall synchronize and display this data in real-time (latency < 2 seconds).

Source	Zulkifl Ahmad (AI/Backend)
Rationale	Cloud storage enables remote monitoring, historical data analysis, and provides the data source for the AI model.
Dependencies	FR-01,FR-02
Priority	High

Identifier	FR-05
Title	Offline Data Buffering and Sync Recovery
Requirement	In the event of Wi-Fi or power interruption, the ESP32 shall store sensor readings in a local buffer (using EEPROM or SPIFFS) for up to 24 hours. Once connectivity is restored, the system shall automatically transmit all buffered data to Firebase in chronological order without data loss.
Source	Zulkifl Ahmad (AI/Backend)
Rationale	Rural farming areas may experience intermittent internet connectivity; this ensures data integrity and continuous monitoring capability
Dependencies	FR-04
Priority	Medium

Module 3: AI/ML Crop Prediction

Identifier	FR-06
Title	AI-based Crop Suitability Recommendation
Requirement	Upon user request from the mobile app, the system shall invoke a pre-trained machine learning model (using Scikit-learn) that takes the latest pH, NPK, Temperature, Humidity, and Rainfall values as input. The model shall output the top 1-3 most suitable crops from the defined list (Wheat, Rice, Maize, Cotton, Tomato) with a suitability percentage. Note: Fertilizer recommendation and yield prediction are explicitly out of scope for v1.0 but are noted for future enhancement.
Source	Zulkifl Ahmad (AI/Backend)
Rationale	To replace guesswork with data-driven science, helping farmers choose the most profitable and sustainable crop for their soil.
Dependencies	FR-04

Priority	High
-----------------	------

Identifier	FR-07
Title	Fertilizer Recommendation based on NPK Deficiency
Requirement	After a crop is recommended or selected, the system shall analyze the soil NPK values against crop-specific optimal ranges. If Nitrogen is low (< 50 mg/kg), the system shall recommend Urea (quantity: 50 kg/acre). If Phosphorus is low (< 30 mg/kg), recommend DAP (40 kg/acre). If Potassium is low (< 40 mg/kg), recommend MOP (30 kg/acre). Multiple fertilizers shall be recommended if multiple deficiencies exist.
Source	Zulkifl Ahmad (AI/Backend)
Rationale	Enables targeted soil nutrient management, reduces fertilizer waste by up to 30%, and lowers farmer input costs through precise application guidance.
Dependencies	FR-06
Priority	High

Identifier	FR-08
Title	Estimated Crop Yield Prediction
Requirement	Upon selecting a recommended crop, the system shall invoke a second ML regression model (Linear Regression or Random Forest Regressor) to predict the expected yield in kg/acre. The model shall take as inputs: selected crop type, historical soil data (average NPK and pH over last 30 days), average temperature, humidity, and expected rainfall. The output shall include a predicted yield value and a confidence interval ($\pm 10\%$).
Source	Zulkifl Ahmad (AI/Backend)
Rationale	Helps farmers make informed decisions about resource allocation, storage planning, loan applications, and financial forecasting for the upcoming harvest season.
Dependencies	FR-06
Priority	Medium

Module 4: Smart Alerts & Notifications

Identifier	FR-09
Title	Real-time Push Notifications for Critical Events
Requirement	The Firebase Cloud Messaging (FCM) service shall send a push notification to the farmer's mobile app when: (a) soil moisture drops below 20%, (b) temperature exceeds 40°C, (c) pH is below 5.5 or above 8.5, or (d) rainfall is detected.
Source	Muneeba Kalsoom (App)
Rationale	Enables the farmer to take immediate corrective action, preventing crop damage and water waste.
Dependencies	FR-04
Priority	High

Identifier	FR-10
Title	Configurable Alert Thresholds
Requirement	The mobile app shall allow the farmer to view and modify the threshold values for each alert type (moisture min/max, temperature min/max, pH min/max) based on their specific crop and regional conditions. Changes made in the app shall sync to the Firebase Realtime Database and be applied to the backend alert evaluation logic.
Source	Muneeba Kalsoom (App)
Rationale	Different crops have different optimal growing conditions; customizable thresholds make the system adaptable to various farming scenarios.
Dependencies	FR-09
Priority	Medium

Module 5: Automatic Irrigation Control

Identifier	FR-11
Title	Threshold-based Automatic Water Pump Activation

Requirement	The ESP32 shall automatically activate a relay connected to a water pump when the soil moisture reading falls below a pre-configured threshold (e.g., 30%). The pump shall deactivate when the moisture reading returns to an upper threshold (e.g., 70%). The farmer shall also have a manual override option in the mobile app.
Source	Aleeza Javed (IoT)
Rationale	Optimizes water usage, reduces manual labor, and prevents over/under-watering.
Dependencies	FR-01
Priority	High

Identifier	FR-12
Title	Manual Override and Remote Pump Control
Requirement	The mobile app shall provide a toggle button to manually turn the water pump ON or OFF, overriding the automatic threshold logic. When manual mode is active, the automatic control shall be temporarily suspended. The farmer shall receive a notification confirming the manual action and can revert to automatic mode with a single tap. The current pump status (ON/OFF) shall be displayed in real-time on the app dashboard.
Source	Aleeza Javed (IoT)
Rationale	Provides the farmer with full control during exceptional conditions (e.g., predicted heavy rain, pump maintenance, or specific growth stage requirements).
Dependencies	FR-04,FR-11
Priority	Medium

Module 6: Mobile App Dashboard

Identifier	FR-13
Title	Real-time Sensor Dashboard
Requirement	The Flutter-based Android mobile app shall display a dashboard that shows the most recent readings for all sensors (pH, NPK, moisture,

	temperature, humidity, light). The dashboard shall update automatically every 2 seconds without requiring a manual refresh. Each sensor's value shall be shown with a color-coded status indicator (Normal/Warning/Critical)..
Source	Muneeba Kalsoom (App)
Rationale	Provides the farmer with an at-a-glance understanding of current field conditions from anywhere.
Dependencies	FR-04
Priority	High

Module 7: User Authentication & Profile

Identifier	FR-14
Title	Secure User Login and Registration
Requirement	The mobile app shall allow new users to register using their email address and a password. Existing users shall log in using their credentials. All authentication processes shall be managed by Firebase Authentication. Each user's sensor data and settings shall be isolated to their own account.
Source	Muneeba Kalsoom (App)
Rationale	Ensures data privacy and security. Allows multiple farmers to use the system without accessing each other's data.
Dependencies	None
Priority	High

Module 8: Reporting & Data Visualization

Identifier	FR-15
Title	Historical Data Trends and Visualization
Requirement	The mobile app shall allow new users to register using their email address and a password. Existing users shall log in using their credentials. All authentication processes shall be managed by Firebase Authentication. Each user's sensor data and settings shall be isolated to their own account.
Source	Muneeba Kalsoom (App)
Rationale	Ensures data privacy and security. Allows multiple farmers to use the system without accessing each other's data.

Dependencies	None
Priority	High

Non Functional Requirements

Usability

USE-1: The mobile application shall require no more than three taps from the home screen to access the main advisory (Crop, Fertilizer, Yield).

USE-2: The mobile app interface shall be available in both English and Urdu languages.

USE-3: 90% of farmers with no prior experience using smart farming apps shall be able to successfully retrieve a crop recommendation within 5 minutes of initial app launch, without external help.

Performance

PER-1: The delay between a sensor reading on the ESP32 and its display on the mobile app dashboard shall not exceed 5 seconds.

PER-2: The AI crop, fertilizer, and yield recommendation pipeline shall return a complete result to the mobile app within 10 seconds of the user request.

PER-3: The automatic irrigation control module shall trigger the relay within 2 seconds of detecting a moisture level below the 30% threshold.

External Interface Requirements

User Interfaces

Mobile App: The system shall provide a graphical user interface (GUI) with support for both dark and light themes. The mobile application shall include the following screens:

- Splash Screen
- Login/Register
- Real-time Dashboard
- AI Advisory Screen (Crop, Fertilizer, Yield)
- Historical Reports Screen
- Profile Screen.

Software Interfaces

Interface between Flutter App and Firebase:

The Flutter application shall communicate with Firebase services using the following Flutter plugins:

- firebase_database
- firebase_auth
- firebase_messaging

Communication shall be performed through Firebase APIs and services.

Interface between Flutter Application and AI Model Server:

The app will send an HTTP POST request to a simple Python Flask/FastAPI server hosting the trained ML models (or call a Firebase Cloud Function which loads the model).

Interface between ESP32 and Firebase

The ESP32 microcontroller shall communicate with Firebase Realtime Database using the Firebase_ESP_Client library and perform HTTP PUT and PATCH operations for updating sensor data.

Hardware Interfaces

Interface between ESP32 and Sensors:

Analog Inputs for Soil Moisture and NPK sensors. I2C/OneWire interface for DHT22 (temperature/humidity). Digital Input for Rain Sensor. UART for any serial-based sensor.

Interface between ESP32 to Relay:

The ESP32 shall use a digital output pin to control a 5V relay module connected to the water pump for automated irrigation

Communications Interfaces

IoT Device (ESP32):

The ESP32 device shall support Wi-Fi IEEE 802.11 b/g/n (2.4 GHz) for connecting to a local wireless access point and transmitting sensor data to Firebase.

Mobile App:

Uses the smartphone's existing cellular (4G/5G) or Wi-Fi connection to communicate with Firebase and the AI model server.

References

1. Flutter Docs <https://docs.flutter.dev>
2. Firebase Docs <https://firebase.google.com/docs>
3. ESP32 Docs (Espressif) <https://docs.espressif.com/projects/esp-idf/en/latest/esp32>
4. Scikit-learn Docs <https://scikitlearn.org/stable/documentation.html>
5. Kaggle Crop Recommendation Dataset
<https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>
6. IEEE 830 SRS Standard <https://standards.ieee.org/ieee/830/1222>
7. Khanna & Kaur (2019) – IoT in Precision Agriculture *Computers and Electronics in Agriculture*, 157, 218–231